Class practical

11.

```
# Top-Down Parser

grammar = {
    "S": [["NP", "VP"]],
    "NP": [["Det", "N"]],
    "VP": [["V", "NP"]],
    "Det": [["the"], ["a"]],
    "N": [["cat"], ["dog"], ["mouse"]],
    "V": [["chased"], ["saw"]]
}

def parse(symbol, words, pos):
    if symbol not in grammar:
        if pos < len(words) and words[pos] == symbol:
            return [pos + 1]
        return []

    results = []

    for production in grammar[symbol]:
        positions = [pos]

        for item in production:
            new_positions = []
            for p in positions:
                new_positions.extend(parse(item, words, p))
            positions = new_positions

        results.extend(positions)

    return results

sentence = input("Enter a sentence: ").lower().split()

if len(sentence) in parse("S", sentence, 0):
    print("Sentence accepted")
else:
    print("Sentence rejected")
```

```
Enter a sentence: the cat chased a dog
Sentence accepted
```

12.

```
# Simple Earley Parser

grammar = {
    "S": [["NP", "VP"]],
    "NP": [["Det", "N"]],
    "VP": [["V", "NP"]],
    "Det": [["the"], ["a"]],
    "N": [["cat"], ["dog"]],
    "V": [["saw"], ["chased"]]
}

def earley_parse(words):
    n = len(words)
    chart = [set() for _ in range(n + 1)]

    # State: (lhs, rhs, dot, origin)
    chart[0].add(("START", ("S",), 0, 0))

    for i in range(n + 1):
        changed = True

        while changed:
            changed = False

            for state in list(chart[i]):
                lhs, rhs, dot, origin = state

                if dot < len(rhs):
                    next_symbol = rhs[dot]

                    # Prediction
                    if next_symbol in grammar:
                        for prod in grammar[next_symbol]:
                            new_state = (
                                next_symbol, tuple(prod), 0, i
                            )
                            if new_state not in chart[i]:
                                chart[i].add(new_state)
                                changed = True
                    else:
                        # Scanning
                        if i < n and words[i] == next_symbol:
                            chart[i + 1].add(
                                (lhs, rhs, dot + 1, origin)
                            )
```

```
                    ,
                else:
                    # Completion
                    for old in list(chart[origin]):
                        a, b, d, o = old
                        if d < len(b) and b[d] == lhs:
                            new_state = (a, b, d + 1, o)
                            if new_state not in chart[i]:
                                chart[i].add(new_state)
                                changed = True

    return ("START", ("S",), 1, 0) in chart[n]


sentence = input("Enter a sentence: ").lower().split()

if earley_parse(sentence):
    print("Sentence accepted")
else:
    print("Sentence rejected")

---------------------- RESTART: C:/USERS/
Enter a sentence: the dog chased a cat
Sentence accepted
```

13.

```
import nltk

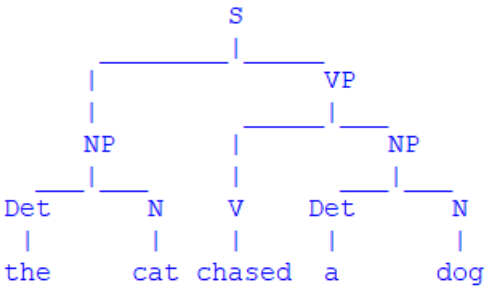
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'cat' | 'dog'
V -> 'chased' | 'saw'
""")

sentence = input("Enter a sentence: ").lower().split()

parser = nltk.ChartParser(grammar)
trees = list(parser.parse(sentence))

if trees:
    for tree in trees:
        print(tree)
        tree.pretty_print()
else:
    print("No parse tree found")
```

```
Enter a sentence: the cat chased a dog
(S (NP (Det the) (N cat)) (VP (V chased) (NP (Det a) (N dog))))
```



14.

```python
# Subject-Verb Agreement Checker

grammar = {
    "S": [["SINGULAR", "VS"]],
    "SINGULAR": [["he"], ["she"], ["it"]],
    "VS": [["runs"], ["eats"], ["plays"]]
}

def check(symbol, words, pos):
    if symbol not in grammar:
        if pos < len(words) and words[pos] == symbol:
            return [pos + 1]
        return []

    result = []

    for production in grammar[symbol]:
        positions = [pos]

        for item in production:
            positions = [
                p2
                for p in positions
                for p2 in check(item, words, p)
            ]

        result.extend(positions)

    return result

sentence = input("Enter a sentence: ").lower().split()

if len(sentence) in check("S", sentence, 0):
    print("Correct subject-verb agreement")
else:
    print("Incorrect subject-verb agreement")
```

```
Enter a sentence: he runs
Correct subject-verb agreement
```

15.

```
import nltk

grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.8] | N [0.2]
VP -> V NP [0.7] | V [0.3]
Det -> 'the' [0.6] | 'a' [0.4]
N -> 'cat' [0.5] | 'dog' [0.5]
V -> 'chased' [0.6] | 'slept' [0.4]
""")

sentence = input("Enter a sentence: ").lower().split()

parser = nltk.ViterbiParser(grammar)
trees = list(parser.parse(sentence))

if trees:
    best_tree = max(trees, key=lambda tree: tree.prob())
    print("Best parse tree:")
    print(best_tree)
    print("Probability:", best_tree.prob())
else:
    print("No valid parse found")
```

```
Enter a sentence: the cat chased a dog
Best parse tree:
(S
  (NP (Det the) (N cat))
  (VP (V chased) (NP (Det a) (N dog)))) (p=0.016128)
Probability: 0.016128000000000003
```

17.

```
import nltk

nltk.download("wordnet")
nltk.download("omw-1.4")

from nltk.corpus import wordnet as wn

word = input("Enter a word: ")

synsets = wn.synsets(word)

if synsets:
    for syn in synsets:
        print("\nSynset:", syn.name())
        print("Definition:", syn.definition())
        print("Examples:", syn.examples())
        print("Synonyms:", [
            lemma.name() for lemma in syn.lemmas()
        ])
else:
    print("No synsets found")
```

```
Enter a word: bank

Synset: bank.n.01
Definition: sloping land (especially the slope beside a body of water)
Examples: ['they pulled the canoe up on the bank', 'he sat on the bank of the river and watched the currents']
Synonyms: ['bank']

Synset: depository_financial_institution.n.01
Definition: a financial institution that accepts deposits and channels the money into lending activities
Examples: ['he cashed a check at the bank', 'that bank holds the mortgage on my home']
Synonyms: ['depository_financial_institution', 'bank', 'banking_concern', 'banking_company']

Synset: bank.n.03
Definition: a long ridge or pile
Examples: ['a huge bank of earth']
Synonyms: ['bank']

Synset: bank.n.04
Definition: an arrangement of similar objects in a row or in tiers
Examples: ['he operated a bank of switches']
Synonyms: ['bank']

Synset: bank.n.05
Definition: a supply or stock held in reserve for future use (especially in emergencies)
Examples: []
Synonyms: ['bank']

Synset: bank.n.06
Definition: the funds held by a gambling house or the dealer in some gambling games
Examples: ['he tried to break the bank at Monte Carlo']
Synonyms: ['bank']

Synset: bank.n.07
Definition: a slope in the turn of a road or track; the outside is higher than the inside in order to reduce the effects of centrifugal force
Examples: []
Synonyms: ['bank', 'cant', 'camber']

Synset: savings_bank.n.02
Definition: a container (usually with a slot in the top) for keeping money at home
Examples: ['the coin bank was empty']
Synonyms: ['savings_bank', 'coin_bank', 'money_box', 'bank']

Synset: bank.n.09
Definition: a building in which the business of banking transacted
Examples: ['the bank is on the corner of Nassau and Witherspoon']
Synonyms: ['bank', 'bank_building']
```

Synset: bank.n.07
Definition: a slope in the turn of a road or track; the outside is higher than the inside in order to reduce the effects of centrifugal force
Examples: []
Synonyms: ['bank', 'cant', 'camber']

Synset: savings_bank.n.02
Definition: a container (usually with a slot in the top) for keeping money at home
Examples: ['the coin bank was empty']
Synonyms: ['savings_bank', 'coin_bank', 'money_box', 'bank']

Synset: bank.n.09
Definition: a building in which the business of banking transacted
Examples: ['the bank is on the corner of Nassau and Witherspoon']
Synonyms: ['bank', 'bank_building']

Synset: bank.n.10
Definition: a flight maneuver; aircraft tips laterally about its longitudinal axis (especially in turning)
Examples: ['the plane went into a steep bank']
Synonyms: ['bank']

Synset: bank.v.01
Definition: tip laterally
Examples: ['the pilot had to bank the aircraft']
Synonyms: ['bank']

Synset: bank.v.02
Definition: enclose with a bank
Examples: ['bank roads']
Synonyms: ['bank']

Synset: bank.v.03
Definition: do business with a bank or keep an account at a bank
Examples: ['Where do you bank in this town?']
Synonyms: ['bank']

Synset: bank.v.04
Definition: act as the banker in a game or in gambling
Examples: []
Synonyms: ['bank']

Synset: bank.v.05
Definition: be in the banking business
Examples: []
Synonyms: ['bank']

Synset: bank.v.05
Definition: be in the banking business
Examples: []
Synonyms: ['bank']

Synset: deposit.v.02
Definition: put into a bank account
Examples: ['She deposits her paycheck every month']
Synonyms: ['deposit', 'bank']

Synset: bank.v.07
Definition: cover with ashes so to control the rate of burning
Examples: ['bank a fire']
Synonyms: ['bank']

Synset: trust.v.01
Definition: have confidence or faith in
Examples: ['We can trust in God', 'Rely on your friends', 'bank on your good education', "I swear by my grandmother's recipes"]
Synonyms: ['trust', 'swear', 'rely', 'bank']

18.

```
# Simple FOPC Parser
# Supported syntax:
# P(x), Q(x,y), !P(x), P(x)&Q(x),
# P(x)|Q(x), forall x: P(x), exists x: P(x)

import re

TOKEN_PATTERN = re.compile(
    r"\s*(forall|exists|[A-Za-z_]\w*|[!&|(),:])"
)

def tokenize(text):
    tokens = TOKEN_PATTERN.findall(text)
    if "".join(tokens).replace(" ", "") != re.sub(
        r"\s+", "", text
    ):
        raise ValueError("Invalid character in expression")
    return tokens

def parse(text):
    tokens = tokenize(text)
    pos = 0

    def expression():
        nonlocal pos
        if pos >= len(tokens):
            raise ValueError("Unexpected end")

        token = tokens[pos]

        if token in ("forall", "exists"):
            quantifier = token
            pos += 1

            variable = tokens[pos]
            pos += 1

            if tokens[pos] != ":":
                raise ValueError("Expected ':'")
            pos += 1

            return (quantifier, variable, expression())

        if token == "!":
            pos += 1
            return ("NOT", expression())
```

```
            return node

        predicate = token
        pos += 1

        if pos >= len(tokens) or tokens[pos] != "(":
            raise ValueError("Expected '(' after predicate")
        pos += 1

        args = []
        while pos < len(tokens) and tokens[pos] != ")":
            args.append(tokens[pos])
            pos += 1
            if pos < len(tokens) and tokens[pos] == ",":
                pos += 1
            elif tokens[pos] != ")":
                raise ValueError("Expected ',' or ')'")

        if pos >= len(tokens):
            raise ValueError("Expected ')'")
        pos += 1

        node = ("PREDICATE", predicate, args)

        while pos < len(tokens) and tokens[pos] in ("&", "|"):
            op = tokens[pos]
            pos += 1
            right = expression()
            node = (op, node, right)

        return node

    tree = expression()

    if pos != len(tokens):
        raise ValueError("Unexpected token: " + tokens[pos])

    return tree

try:
    formula = input("Enter a logical expression: ")
    print("Parsed expression:")
    print(parse(formula))
except (ValueError, IndexError) as error:
    print("Invalid expression:", error)
```

```
Enter a logical expression: forall x: P(x)&Q(x)
Parsed expression:
('forall', 'x', ('&', ('PREDICATE', 'P', ['x']), ('PREDICATE', 'Q', ['x'])))
```

19.

```python
import nltk

nltk.download("wordnet")
nltk.download("omw-1.4")

from nltk.corpus import wordnet as wn
from nltk.wsd import lesk

sentence = input("Enter a sentence: ")
word = input("Enter the ambiguous word: ")

sense = lesk(sentence.split(), word)

if sense:
    print("Word:", word)
    print("Selected sense:", sense.name())
    print("Definition:", sense.definition())
    print("Examples:", sense.examples())
else:
    print("No suitable sense found")
```

```
Enter a sentence: he deposited money in the bank
Enter the ambiguous word: bank
Word: bank
Selected sense: savings_bank.n.02
Definition: a container (usually with a slot in the top) for keeping money at home
Examples: ['the coin bank was empty']
```

20.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Python is a programming language",
    "Machine learning uses Python",
    "Natural language processing analyzes text",
    "Python is useful for data science"
]

query = input("Enter search query: ")

vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(documents)
query_vector = vectorizer.transform([query])

scores = cosine_similarity(
    query_vector, tfidf_matrix
).flatten()

ranking = scores.argsort()[::-1]

print("\nRanked Documents:")

for index in ranking:
    if scores[index] > 0:
        print(
            f"Document {index + 1}: {documents[index]}"
        )
        print(f"Similarity: {scores[index]:.4f}\n")
```

```
---------------------- RESTART: C:/Users/Ambuseiva/1
Enter search query: python programming

Ranked Documents:
Document 1: Python is a programming language
Similarity: 0.7287

Document 2: Machine learning uses Python
Similarity: 0.1860

Document 4: Python is useful for data science
Similarity: 0.1531
```